🚗 Driving School AI Tool

🧭 Project Overview

The goal of the project is to make an app that assists driving schools. The idea is to use the back camera of a smartphone to record the road while students are driving, and give advice on their driving. We plan to accomplish this with a combination of an on-device model and a backend model. The on-device model will use the smartphone footage to predict moments when bad driving might be happening. These candidate clips get sent to a backend model, which will determine what specific issues were present in the clip, if any.

To create and test these models, we need to use a combination of datasets. We will use the **template code in templatev4.ipynb** to interact with the data to ensure that all the data is in the same format and it is easy to work with.

🧩 HOW TO USE THE TEMPLATE CODE

We will use pandas **dataframes** to store all our data. We want to keep as much data as possible in these dataframes, and **only** use external data storage for large things like videos and images. We will make a dataframe for each dataset we use, and each row in this dataframe will correspond to a driving clip. See **nexar_dataframe.ipynb** for an example of a properly formatted dataframe. The dataframe will have the following columns:

1. filepath: name of the video file
2. source: where the actual video clip is stored. So far the only supported options are "huggingface" and "googledrive"
3. event_time: The amount of seconds into the clip where something happens (NA if nothing happens in the clip)
4. event_description: A description of the event that happened at event_time
5. split: If the dataset creators split the data into train and test, indicate that here
6. other_info: Any other info about the clip specific to your dataset. Try to include everything available so that we don't leak data.

📊 Here's what a properly formatted dataframe looks like

 image

We will store all video clip files externally on Google Drive, Hugging Face, or any other suitable platform. Working with data that is external is difficult, so to make it easy we can use template code that was made to easily work with this external data and use the dataframe format described above.

All the relevant template code is in **templatev4.ipynb**. This Jupyter notebook is split into 3 sections:

1. Imports
2. template code
3. example usage

The **imports** and **template code** sections should NOT BE CHANGED at all **(make sure that AI code writing tools don't modify this code)**. Simply run those sections of the code. The template code gives 3 functions to easily use the data.

🧱 Template Functions

1. **load_video(row):** takes in a row of a dataframe corresponding to a clip, and retrieves the video clip. Depending on the source column, it will pull the video from the correct place and put it in a folder called temp_videos. **The function returns a full path to the video clip.**
2. **delete_video(video_path):** takes in a video clip path and deletes the video from your device.
3. **upload_to_drive(file_path, folder_id):** Uploads a video clip file into Google Drive. file_path is the local path to the clip on your device. folder_id is the folder_id of the folder on google drive you want to upload your clip into. This function returns the file ID of the clip on the drive.

📚 Example usage of the template code




☁️ HOW TO USE GOOGLE DRIVE

We have a 2TB shared Google Drive account to store video clips, images, and other large data. Make sure to keep everything else locally in dataframes and push them to the repo.

Firstly, should you even store your video clips on the drive? If your video clips are already stored somewhere that's easy to stream from, then there's no need to stream it from google drive, we can stream it from the source directly. The Nexar dataset is on HuggingFace, and the load_video function supports streaming each video directly from HuggingFace. If you have some other non-HuggingFace source you want to stream clips from, add that functionality to the load_video function and let the team will know. However, if you have to download your data, then it's a good idea to upload it to the google drive.

🔑 Shared Google Drive Login

email:drivingtoolproject@gmail.com

password:sigaida123!

DON'T SHARE THIS WITH ANYONE.
Here's what the drive looks like:



Every dataset should have its own folder. Don't leave files lying around. In Google Drive, every folder and file has an ID, and we need to use these IDs to both upload and stream data files.

🆔 How to Get Folder and File IDs

Here's how to get a folder's ID (you need to do this). The highlighted part of the URL is the ID of the test folder:

Folder ID Example



You don't necessarily need to manually check file IDs, but here's how to do it if you need to. Select the option "open the file in a new tab", and then the highlighted part of the URL is the file ID:

File ID Example

 

💾 Template Functions for Google Drive

The template code in templatev4.ipynb has functions to upload and stream files from the Google Drive. **upload_to_drive(file_path, folder_id):** Uploads a video clip file into Google Drive. file_path is the local path to the clip on your device. folder_id is the folder_id of the folder on google drive you want to upload your clip into. This function returns the file ID of the file on the drive, which is the only way to access the file, so you should store this file ID in a dataframe so it can be used to retrieve the file later.

Example of uploading a file to Google Drive



Example of loading a video from Google Drive

You should use the load_video function from the template code to do this, with the column source = "googledrive" and keep the fileID in the dictionary in the other_info column. Don't forget to delete the file when you're finished with it 😃



When running the Google Drive code, you will be prompted to log in to Google Drive in your browser. Log in to **drivingtoolproject@gmail.com**.

🤖 On-device model

The goal of this model is to predict when bad driving might be happening. The on-device model should be light enough to run directly on a smartphone for a long period of time. To be light enough, this model will analyze video frames and not the entire video itself. The current plan is to use DINOv3 (a vision transformer) to make the prediction of when bad driving might be happening. We plan to develop this model in several stages, starting simple and becoming more thorough over time.

🧩 Stage 1

Split each clip into frames that are 1 second apart. For this stage, we can use the Nexar dataset, which has clips of driving with crashes. Any frame before a crash will get a label of 0, and any frame after the crash will

get a label of 1. Then, we can train a DINOv3 head to predict these labels.

🧩 Stage 2

When driving, everything is stationary except for the vehicles (and possibly pedestrians, traffic light states, etc.). In stage 1, the frames that the DINOv3 model receives have no information about the velocity and speed of the vehicles in the scene, making it difficult to make an informative prediction. To fix this, we need to figure out the velocity and acceleration of user's car and all other cars. smartphone GPS can get the user's car speed with an accuracy of +- 1-2mph. The next step is figuring out the other vehicles' relative velocity to the user's car. The same DINOv3 backbone can be used, with small specialized segmentation and depth heads, to segment each vehicle in the frame and figure out their distance. This can be used to calculate the other vehicles' relative velocity. Finally, we can integrate all the vehicles' velocities and accelerations into the frame data to help the DINOv3 model make a better prediction.

🧩 Stage 3

We have not been able to find a dataset of somewhat bad driving. We have datasets of collisions and datasets of good driving, but none with clips of somewhat bad driving, which is what we aim to predict. Consequently, we will likely have to do the majority of pretraining on good driving data. We can accomplish this with a self-supervised approach. DINOv3 already has a small head to predict a global vector that represents the entire scene. Generally, in "good driving", the stuff that happens is very predictable. We can use a self-supervised approach where we train the DINOv3 model to predict future scene vectors. During inference, when the predicted vectors differ from the observed vectors, it may be an indication of bad driving.

🧠 Backend model

The goal of this model is to take candidate driving clips and figure out what kind of bad driving is happening, if any. Vision Language Models(VLMs) are the best candidate for this, because they are the bleeding edge at video analysis. Some candidates for modern VLMs include Qwen3-VL and Gemma 3, but there could definitely be other better options too. These models are generally very big, often ~100B parameters, and therefore likely won't be able to run locally. Realistically, for now, we will likely need to use API calls or cloud instances to use these VLMs. Once we pick a VLM, we will develop our backend model workflow in several stages.

🧩 Stage 1

For the first stage, we just want to see if these VLMs work at all. To do this, we can simply feed in video clips and prompt the VLM to "explain whats happening in this clip", and play around with other prompts to see the VLM's capability.

🧩 Stage 2

Once we have a working VLM, we want the VLM to predict if something bad is happening in the clip and describe what specifically is happening. We should develop a prompt to get this information. Then, we

should evaluate the VLM's performance by comparing the model output to the actual clip annotations.

🧩 Dataset synthesis

Additionally, we may want to use the backend model to help us synthesize a dataset. We currently don't have a dataset of somewhat bad driving, so we could potentially use a VLM to scrub over a large amount of general driving data to find instances of bad driving. That dataset would be helpful for training the on-device model.